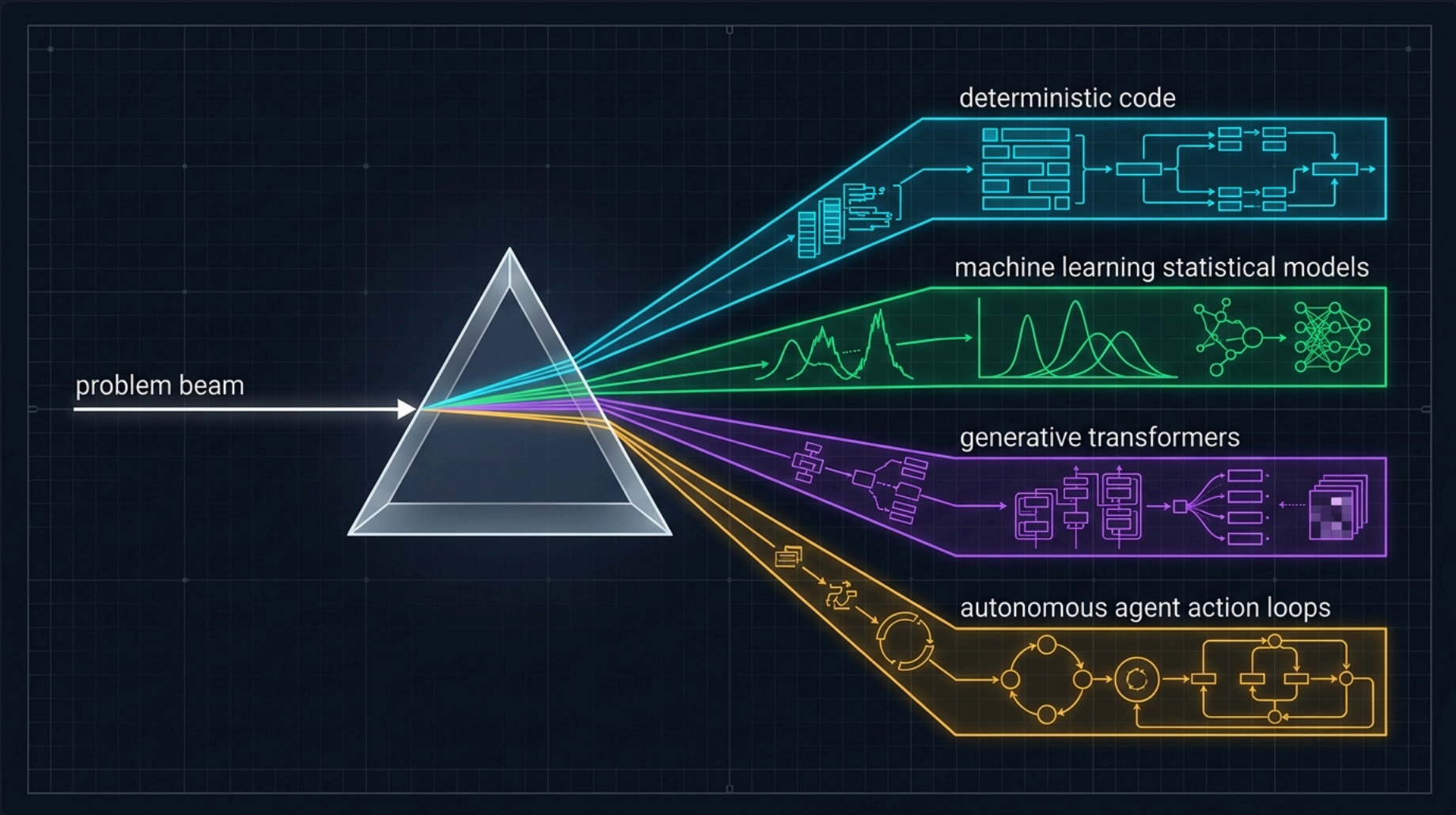


AI Is Not One Thing: The Problem Spectrum

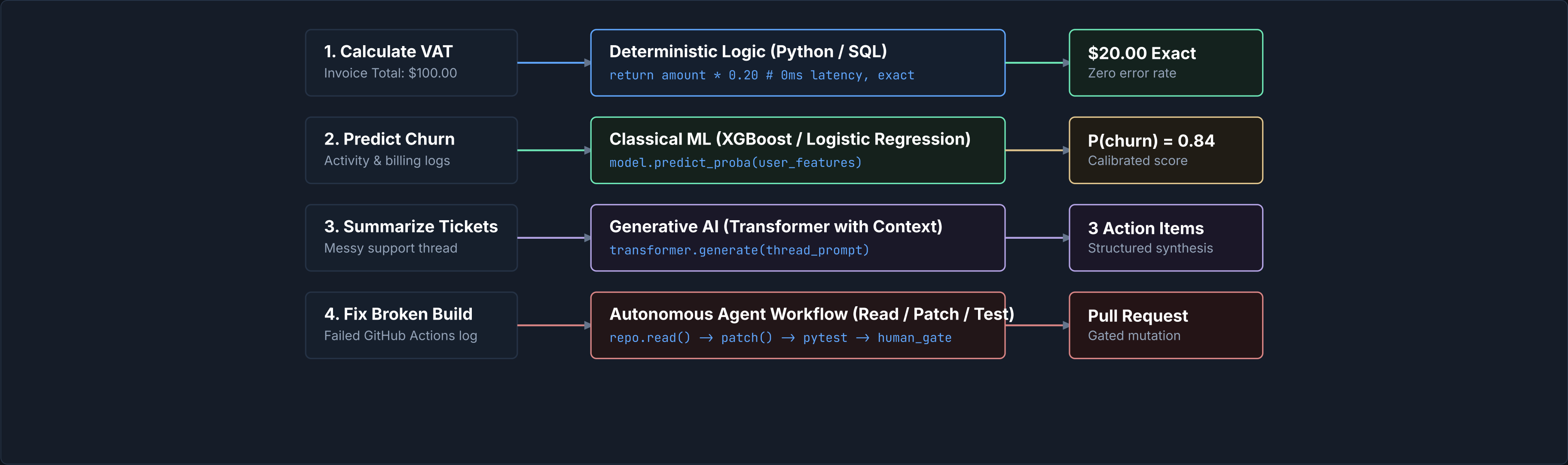
The primary engineering skill is classification: identify the problem type before choosing a technique or model.



Core Axiom: Classify before choosing a model. AI is not a universal hammer.

Four Real Tasks: Four Distinct Architectures

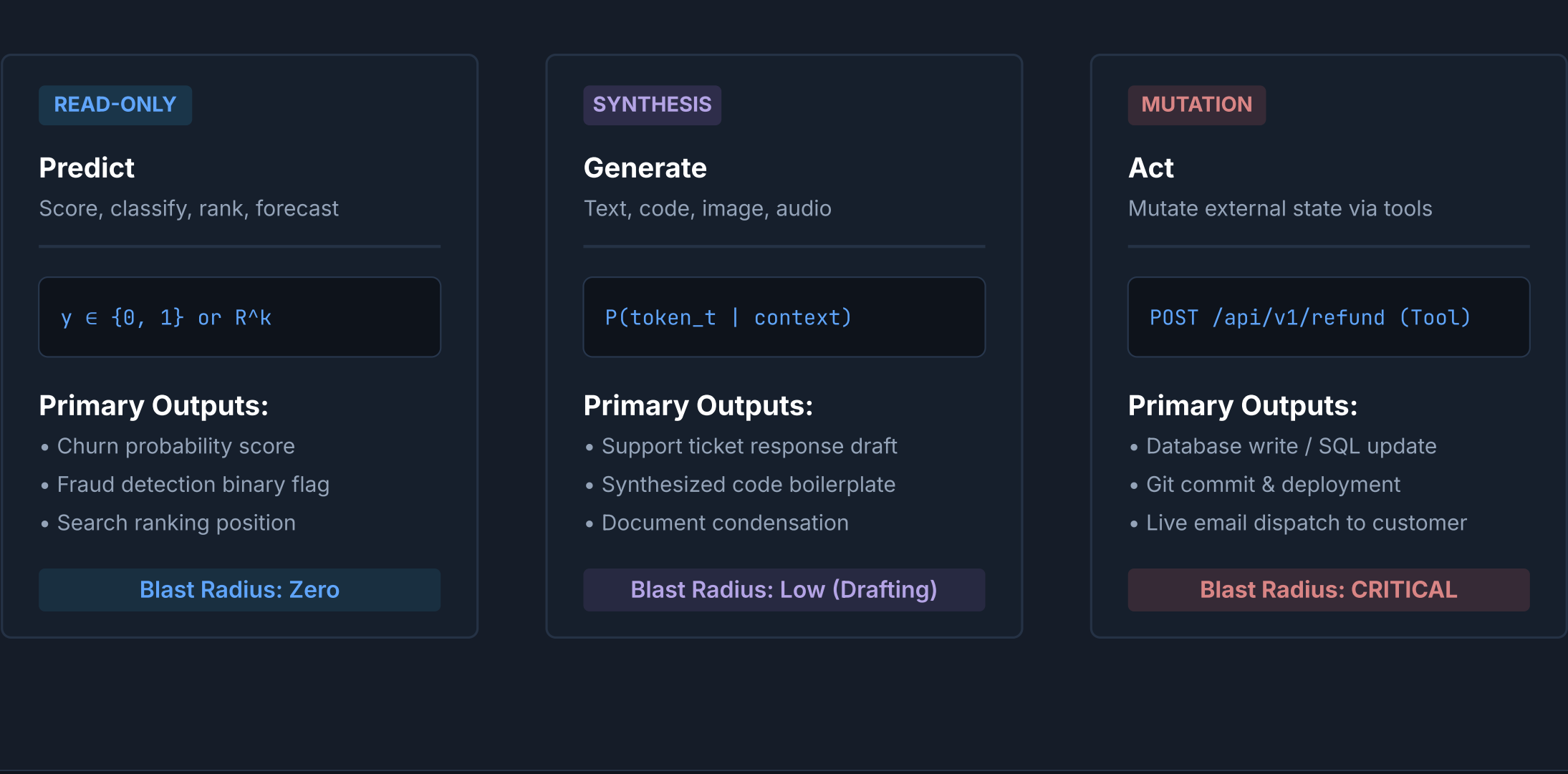
Forcing all workloads into an LLM degrades reliability and spikes cloud latency and costs.



Rule: Never use a probabilistic model where deterministic arithmetic exists.

Predict, Generate, Act: The Three Primitives

Separating system capabilities across these three verbs determines your architecture and blast radius.



Takeaway: Moving from Generate to Act requires least privilege, sandboxes, and human approvals.

Automation Is Not Automatically AI

Deterministic code remains the engineering default when business rules are stable and well-defined.



Principle: AI adds non-zero error rates and latency. Use it only where rules cannot be hard-coded.

Traditional Programming vs Machine Learning

Who derives the rules: the human engineer writing conditions, or the optimization algorithm fitting examples?

TRADITIONAL PROGRAMMING: HUMAN ENCODES THE RULES



MACHINE LEARNING: ALGORITHM DERIVES THE RULES

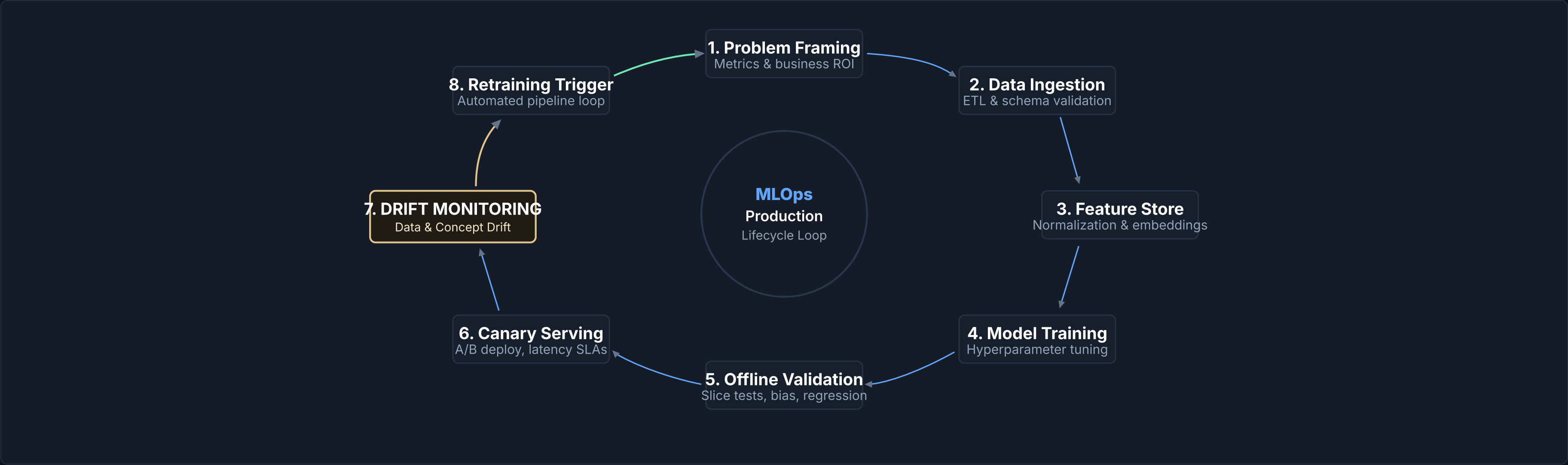


Inference Phase: **New Data** is fed into the **Learned Model (W)** to compute **Predictions** without rewriting code.

Takeaway: When writing rules by hand is impossible (e.g. computer vision, spam), ML fits the rules from data.

A Model File Is Not a Product: The MLOps Lifecycle

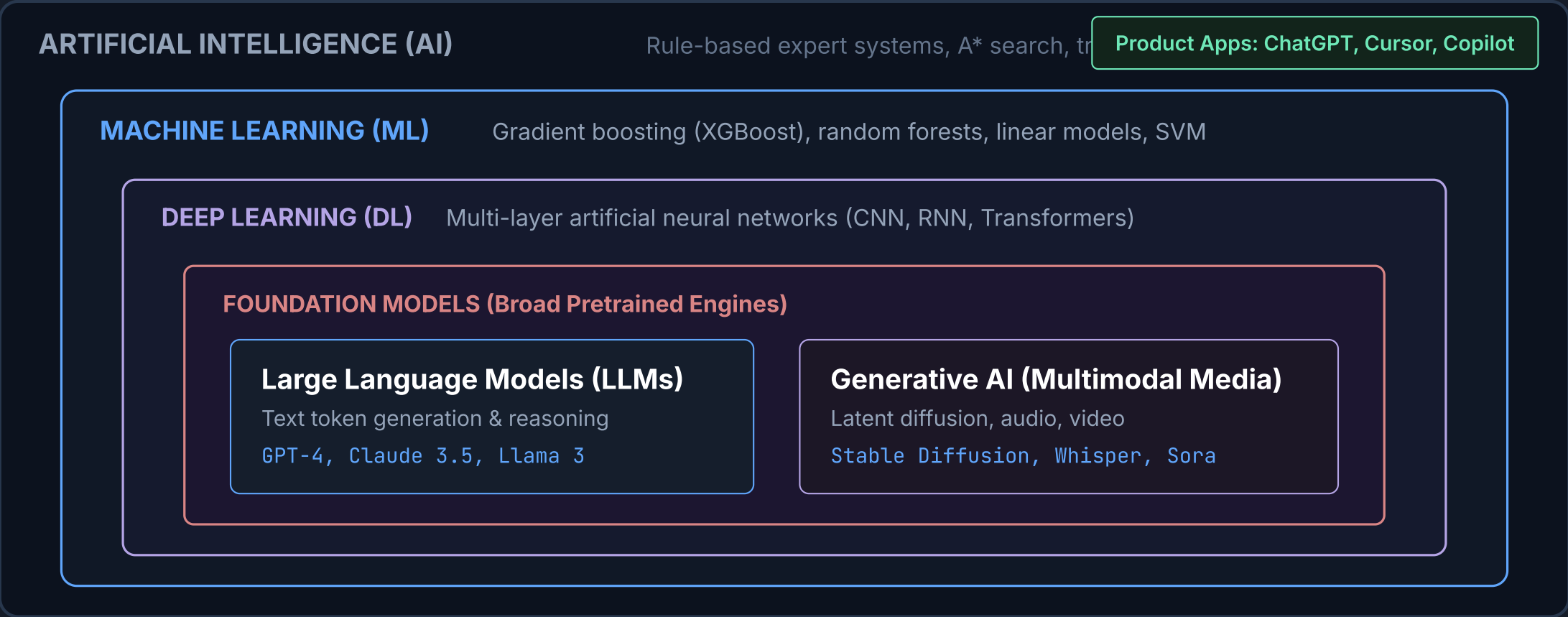
Training a weights file is only 15% of the total engineering effort. Production systems require continuous feedback.



Takeaway: Production systems decay when environments change. Monitoring and retraining are mandatory.

Vocabulary Map: Precise Technical Boundaries

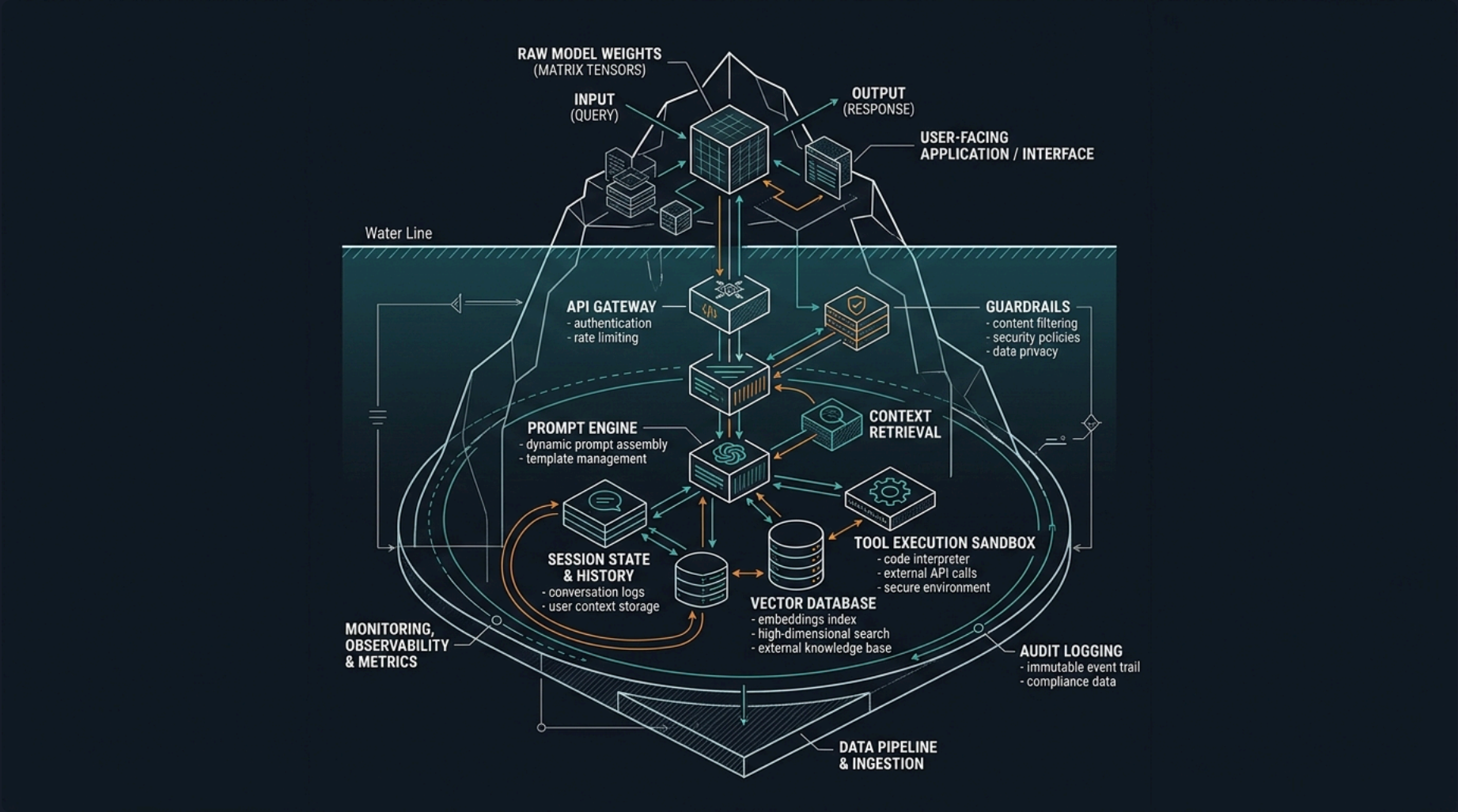
Avoid category errors: AI is broader than ML, Deep Learning is broader than LLMs, and ChatGPT is a product wrapper.



Takeaway: LLMs are one subset of Foundation Models, which are a subset of Deep Learning and ML.

Model ≠ Product: The Architecture Iceberg

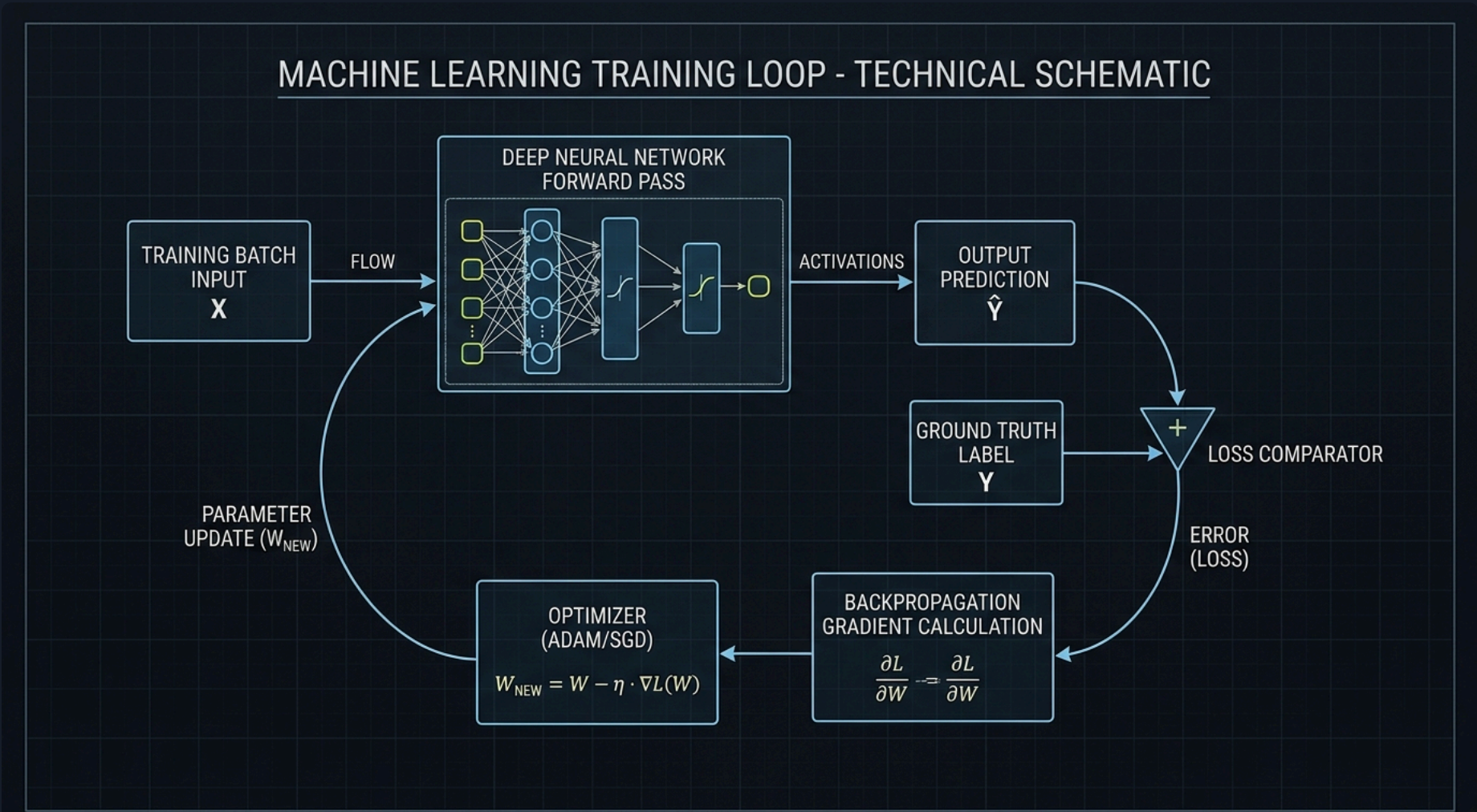
Raw weights are stateless matrix tensors. Over 80% of an AI product is runtime infrastructure, memory, and governance.



Takeaway: A model alone has no memory, tools, or guardrails. The product is the scaffolding.

The Training Loop: How Weights Actually Learn

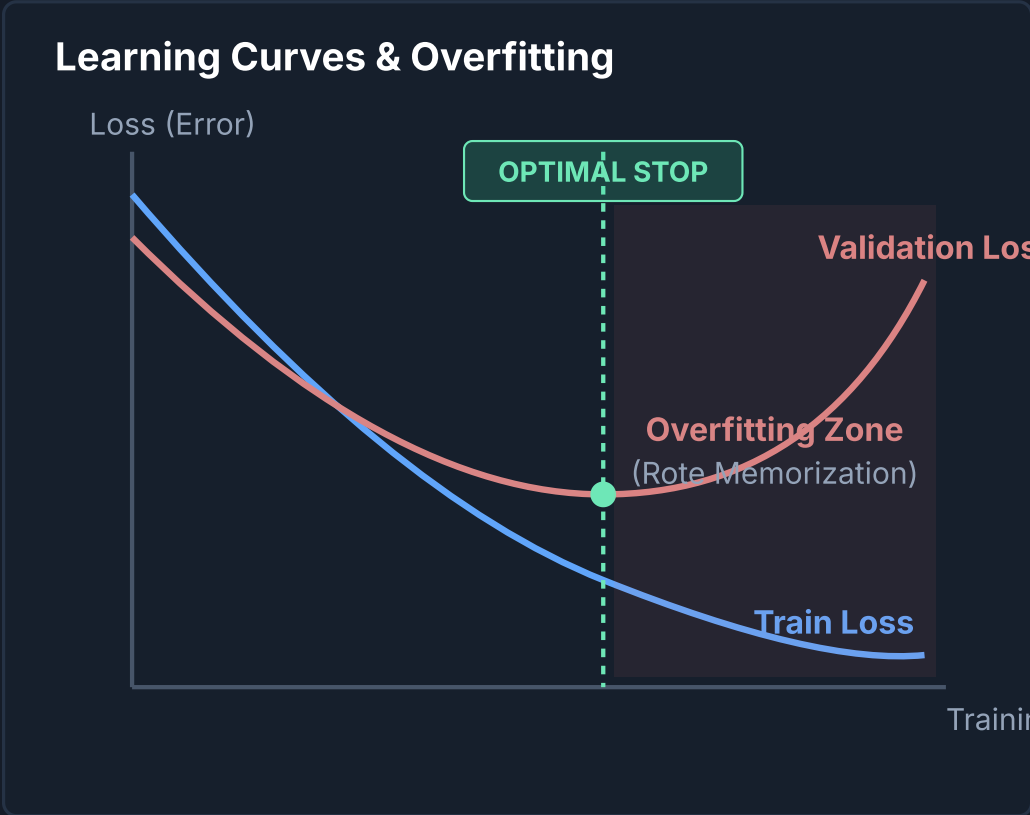
Neural networks optimize weights iteratively: Forward Pass → Loss Calculation → Backpropagation → Gradient Update.



Distinction: Training mutates parameters offline at high cost. Inference freezes weights for sub-second serving.

Generalization Beats Memorization: The Overfitting Trap

High training accuracy is trivial to fake by rote memorization. Models must generalize to unseen test distributions.



Evaluation Rigor: 70 / 15 / 15 Split

1. Training Set (70%)

The textbook. Gradients and weight updates are directly computed here.

2. Validation Set (15%)

The practice exam. Tunes hyperparameters and halts training at optimal loss.

3. Test Set (15%): Sealed Vault

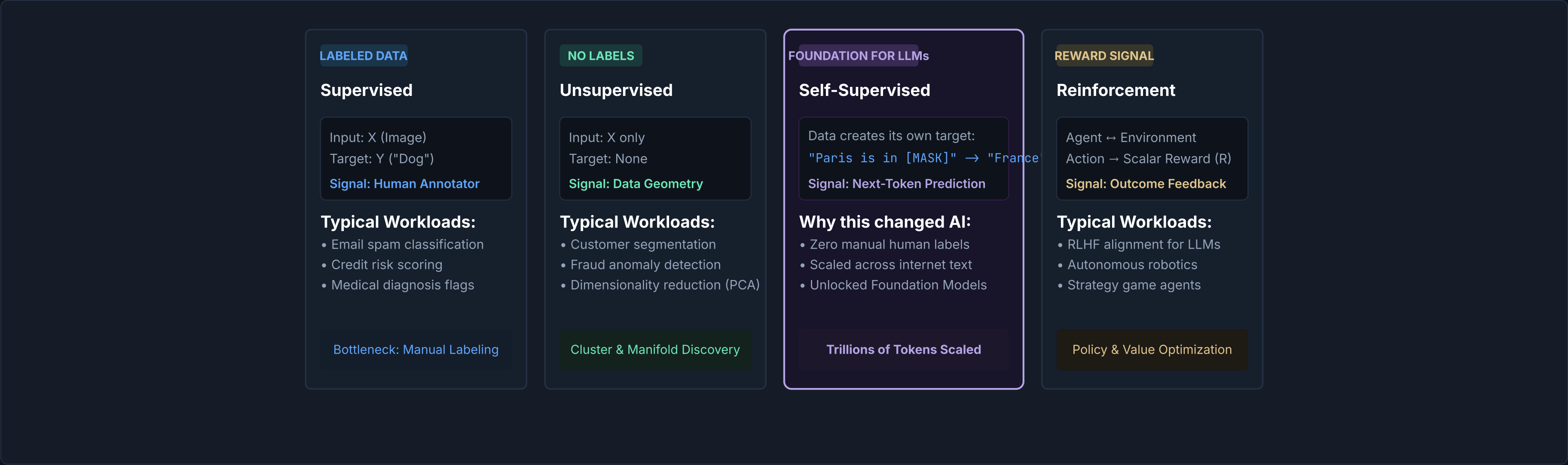
Evaluated strictly once before production deployment. Measures true generalization.

Analogy: Memorizing last year's exam answers fails when variables change.

Rule: Never evaluate an ML model on data it touched during gradient updates.

Four Learning Patterns: The Origin of the Signal

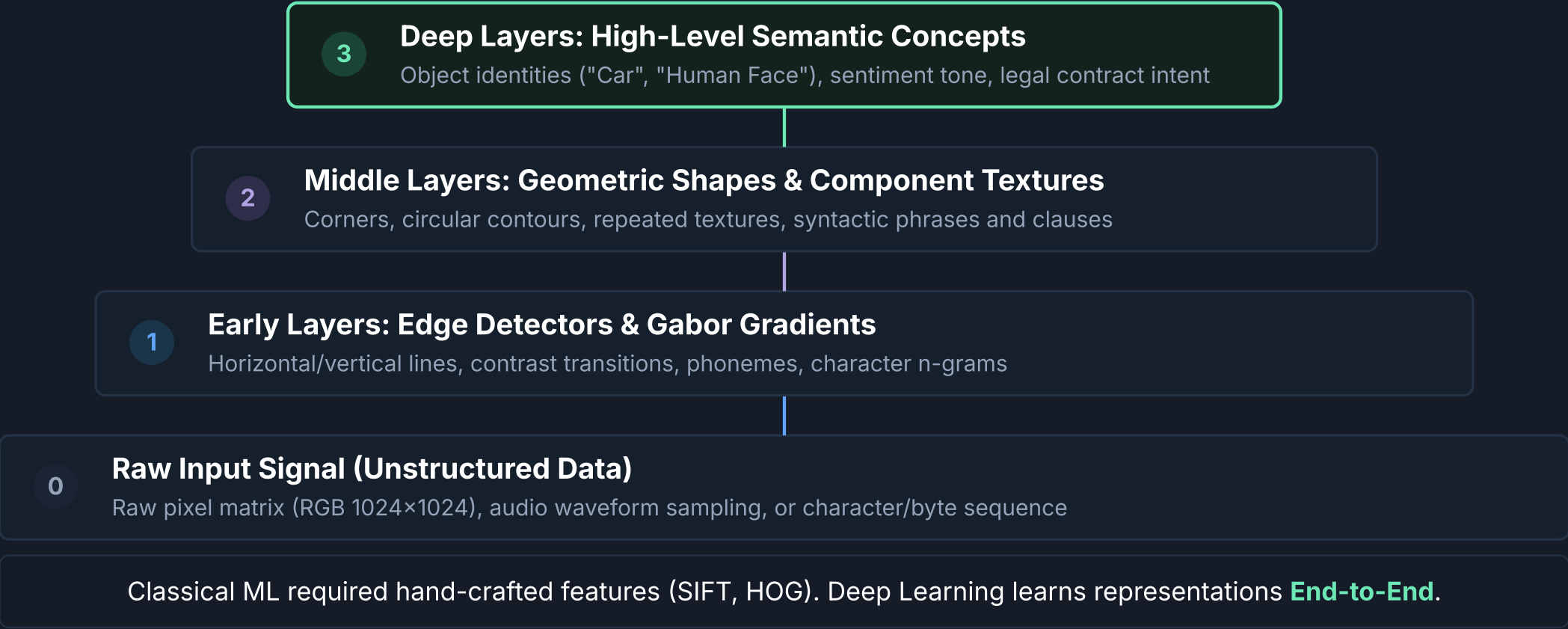
The major branches of machine learning differ by how the optimization algorithm acquires its training signal.



Takeaway: Self-supervised masking eliminated the human labeling bottleneck and unlocked modern LLMs.

Deep Learning: Hierarchical Representation Learning

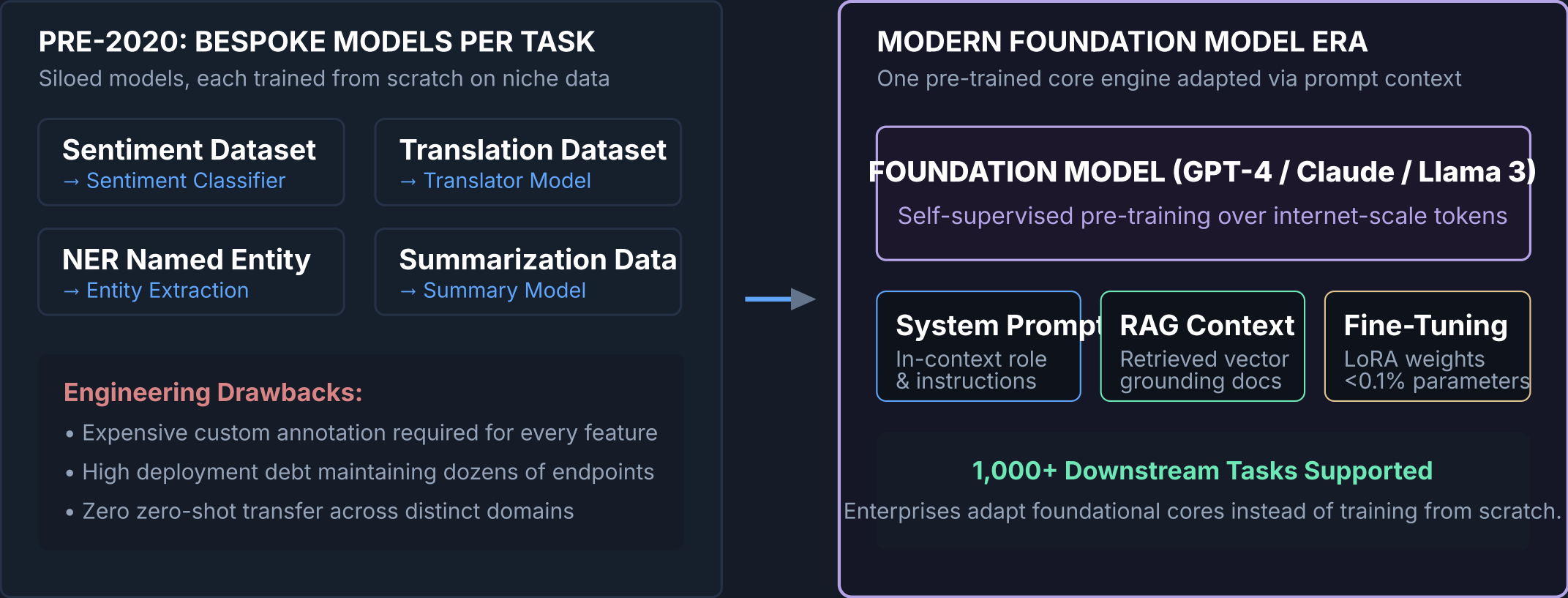
Neural networks construct representations automatically, from low-level pixels to high-level semantic concepts.



Takeaway: Deep networks automate feature engineering by building stacked mathematical abstractions.

Foundation Models: The Reuse Paradigm Shift

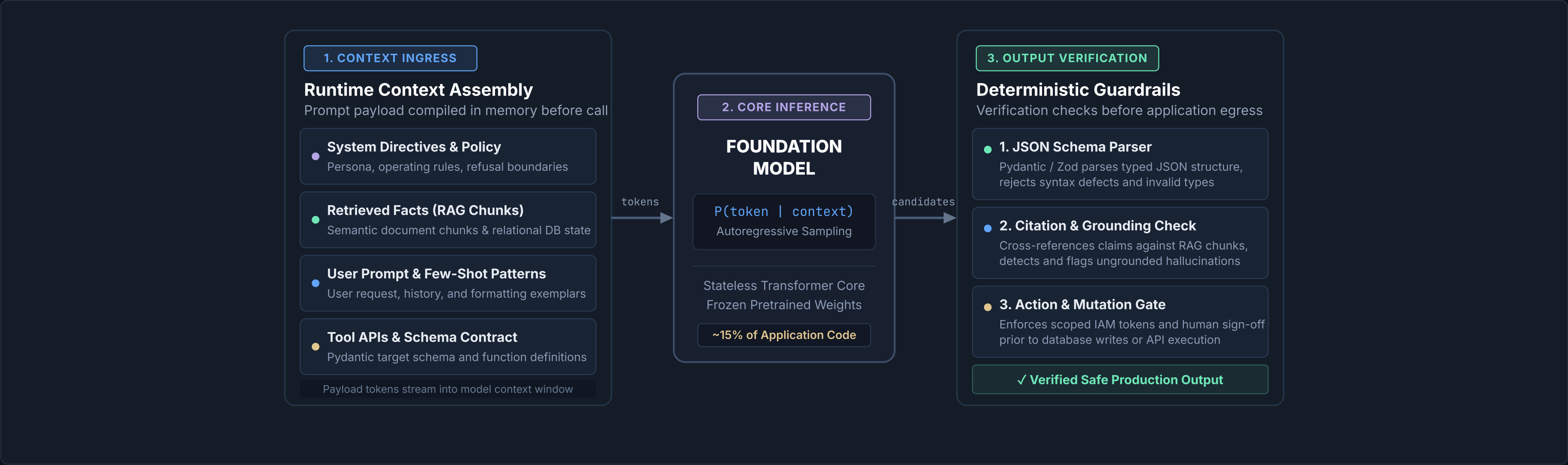
Moving from one bespoke model per task to adapting a massive general-purpose foundation model via context.



Economics: Training frontier models costs \$100M+. Adapting them with RAG and context costs pennies.

Application Context Surrounds the Model

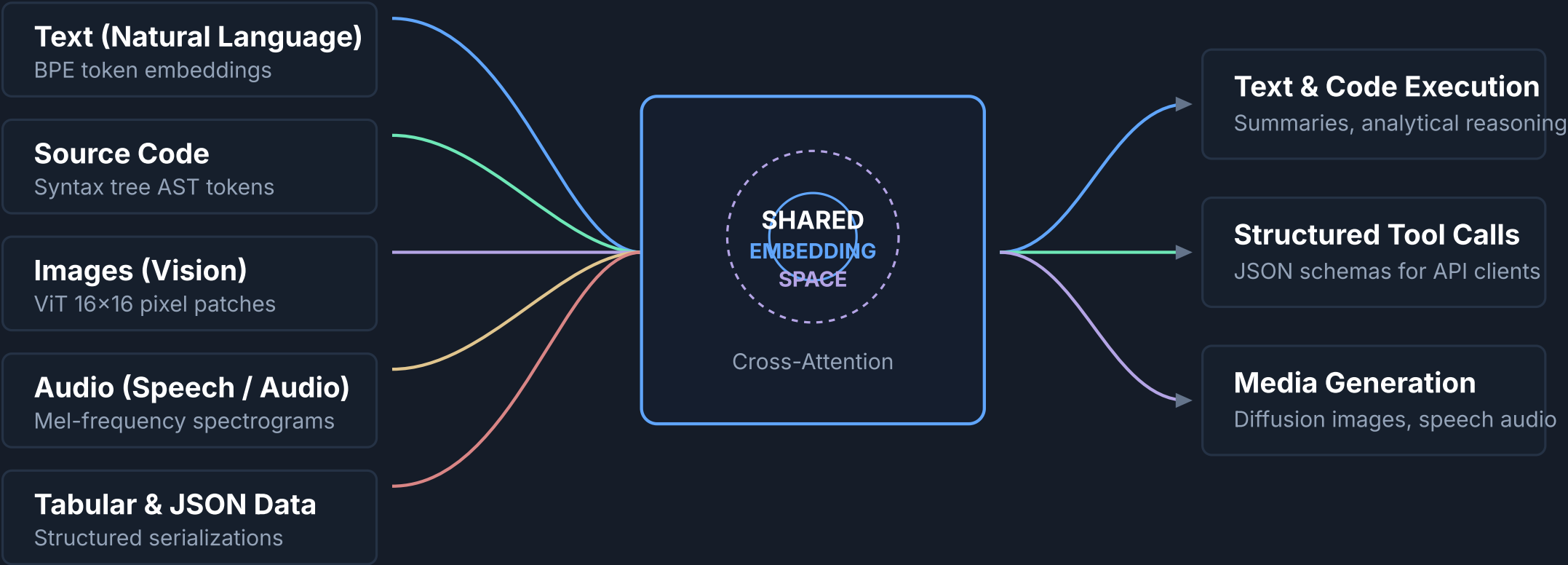
System reliability is achieved by strict prompt assembly on ingress and deterministic schema validation on egress.



Axiom: Garbage in, hallucination out. Context engineering governs system accuracy.

Generative AI Is Broader Than Text Chat

Modern multimodal transformers project disparate media into a shared latent vector space for unified reasoning.



Takeaway: Multimodality means pictures, code, and text live in the exact same vector space.

Fluent Is Not Grounded: Why LLMs Hallucinate

Language models optimize for linguistic plausibility, not factual database integrity.

SOURCE OF TRUTH

Relational Database (PostgreSQL)

Exact memory pointer lookup

SELECT balance FROM accounts WHERE id=42;

Result: **\$1,450.00** (or NULL if missing)

Integrity Guarantees:

- Fact truth value = 1.0 (binary certainty)
- Returns NULL if record does not exist
- Zero probability of fabricating money

PROBABILISTIC SYNTHESIS

Autoregressive Language Model

Predicts next most likely token

"The balance for user 42 is..."

P(token) → **"\$2,500.00"** (Fluent, plausible, false!)

Vulnerability Surface:

- Optimizes for syntactic smoothness, not truth
- Confidently invents numbers without context
- Critical production risk without strict RAG grounding

Takeaway: Plausible structure is useful. Plausible facts without grounding are dangerous.

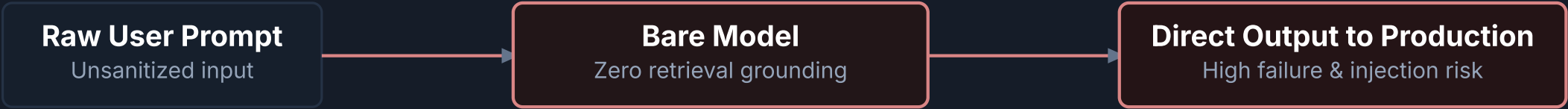
Rule: Never ask an ungrounded LLM for stored business facts. Retrieve the fact first.

Lecture 01 • Section 8

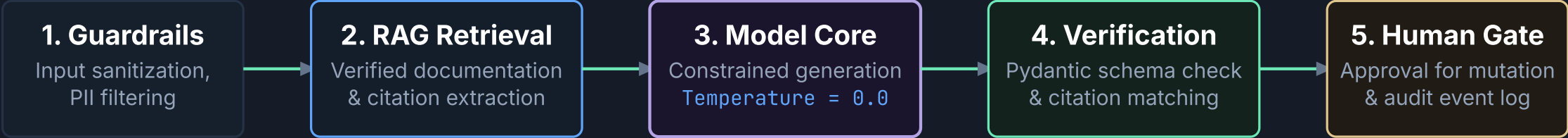
Fragile Workflow vs Production-Grade Architecture

Production-grade systems surround the probabilistic model with deterministic validation and guardrails.

FRAGILE WORKFLOW (TOY DEMO)



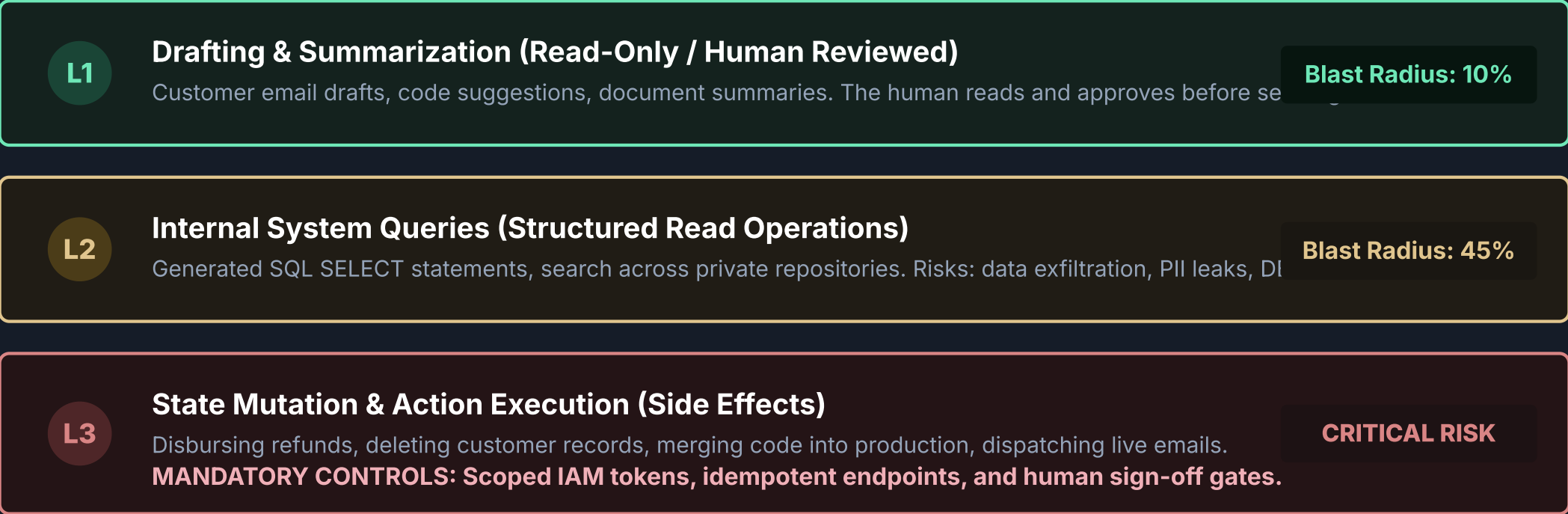
PRODUCTION-GRADE WORKFLOW (ENTERPRISE STANDARD)



Takeaway: In production, the model is only a computational step, wrapped by deterministic validation.

Generation ≠ Execution: The Blast Radius Spectrum

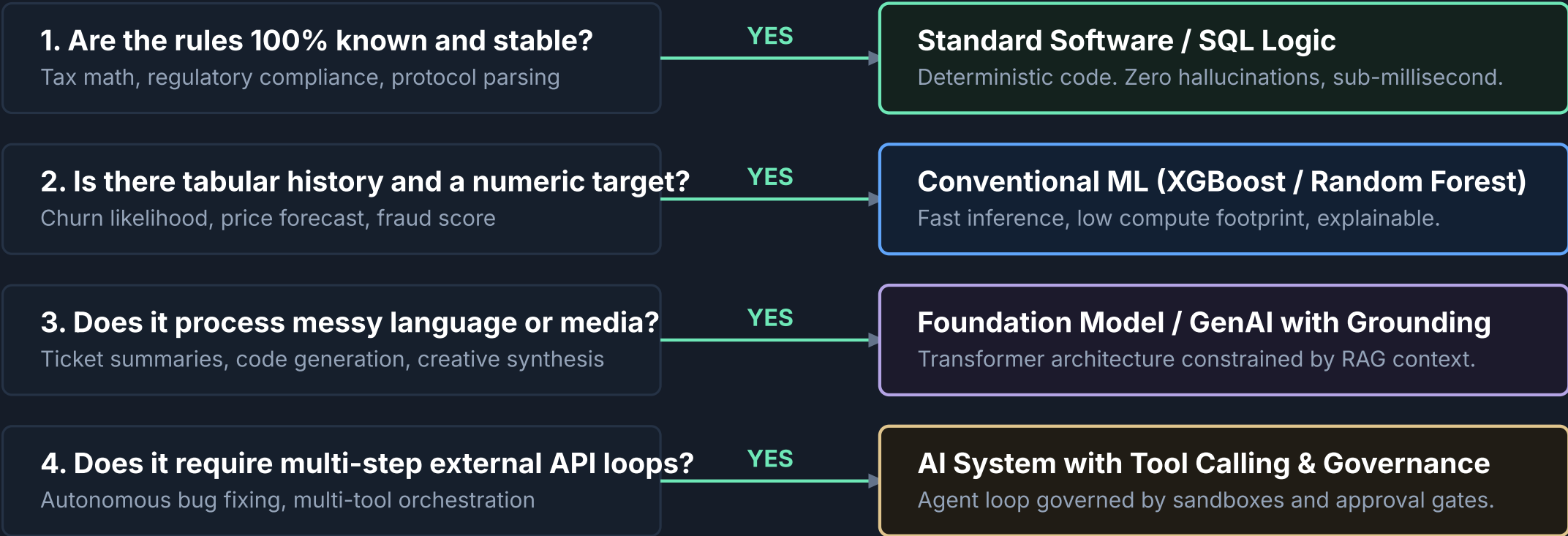
Drafting text carries near-zero impact. Mutating external systems via tools introduces critical operational risk.



Takeaway: As blast radius grows, autonomous agent authority must be constrained by engineering gates.

Technology Selection Rule: The Engineering Compass

Select the component based on the structural nature of the problem, not marketing buzzwords.



Takeaway: Use the simplest adequate technique that solves the business requirement.

Ask for the Source of Truth: Prevent Anti-Patterns

Do not ask the model to invent stored facts. Route queries to their authentic system of record.

WHAT THE SYSTEM NEEDS

User balance, order status

Transactional exact values

Company return policy

Corporate regulatory document

Customer churn likelihood

Historical pattern correlation

Courteous customer reply

Synthesizing facts into prose

AUTHENTIC SOURCE OF TRUTH

Relational Database

PostgreSQL / MySQL tables

Policy Document / Wiki

Verified knowledge base

Historical Log Dataset

Structured analytics warehouse

Retrieved Context Payload

DB fact + verified policy snippet

ENGINEERING ACTION

Query It

```
SELECT balance FROM db
```

Retrieve It

```
rag.retrieve(chunks)
```

Predict It

```
model.predict_score()
```

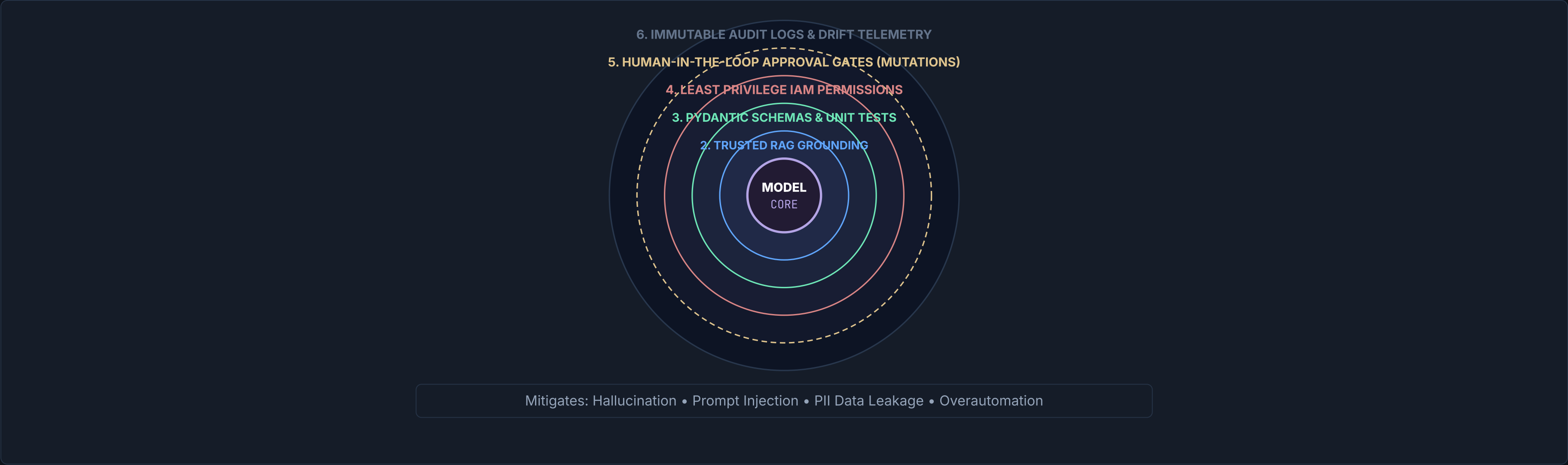
Synthesize It

```
llm.generate(context)
```

Takeaway: First query the source of truth, then inject facts into the model context for drafting.

Controls Belong Around the Model: Defense-in-Depth

Risk management is an architectural discipline. Models are surrounded by concentric layers of deterministic checks.



Takeaway: Enterprise safety is engineered outside the model weights, through layered perimeter defenses.

Mental Model for the Course: Task → Technique → Controls

Four foundational engineering axioms established in Lecture 01, and a preview of technical mechanics in Lecture 02.

FOUR CORE ENGINEERING AXIOMS

1

Start from the task, not the model

Classify the problem space before selecting libraries.

2

Use the simplest adequate technique

Ordinary code beats probabilistic models in speed and cost.

3

Wrap models with context and checks

Grounding facts and Pydantic schemas guarantee integrity.

4

Treat actions as production changes

Tool execution requires least privilege and human approval.

PREVIEW: LECTURE 02 (LLM FUNDAMENTALS)

From Architecture to Mechanics

Deep technical dive into token generation and retrieval:

1. Tokenization: Byte-Pair Encoding (BPE) internals
2. Self-Attention mechanism & context window bottlenecks
3. Production Prompt Engineering & System Directives

4. Hands-on RAG pipeline implementation in Python